Trabalho 2: O Cavalo e os Hiperpulos

Guilherme Guimarães Santos

Escola Politécnica, PUCRS

Algoritmos e Estruturas de Dados II, Turma 320

Prof. João Batista Souza de Oliveira

11 de novembro de 2026

Resumo

Este trabalho trata de um desafio de xadrez com regras diferentes das habituais: em um tabuleiro quadrado cujas bordas se encostam, cada casa traz um dígito de 0 a 9 que aumenta o pulo em L do cavalo, e o cavalo deve ir da casa C até a saída S no menor número de movimentos. O tabuleiro é modelado como um grafo, com uma casa por vértice e um pulo por aresta, e o problema é resolvido com busca em largura (BFS), que encontra o menor caminho quando todas as arestas custam o mesmo. O custo da BFS é proporcional ao número de casas, $\Theta(N^2)$ para um tabuleiro $N \times N$, e as medições confirmam: dobrar N multiplica o tempo por cerca de quatro. Os oito tabuleiros fornecidos, de 40×40 a 1500×1500, foram resolvidos com 4 a 43 movimentos, cada um em no máximo 57 ms, e as respostas foram conferidas por uma segunda implementação, por busca exaustiva e pelo algoritmo de Dijkstra.

1 Introdução

O problema tratado neste trabalho é um desafio de xadrez com regras diferentes das habituais. Dado um tabuleiro quadrado de lado N, com um dígito de 0 a 9 em cada casa, e um cavalo em uma casa marcada com C, o objetivo é levar o cavalo até uma casa de saída, marcada com S, no menor número possível de movimentos.

Duas regras tornam o problema diferente do xadrez comum. A primeira é que o dígito das casas aumenta o pulo do cavalo: o dígito 0 corresponde ao pulo normal de xadrez, em forma de L, e dígitos maiores esticam o L, como mostra a Figura 1. A segunda é que as bordas do tabuleiro se encostam: se o cavalo sai pela direita, entra pela esquerda, e se sai por cima, entra por baixo. Por isso diz-se que o tabuleiro é *toroidal*.

O enunciado traz um tabuleiro de exemplo, com 10 linhas e 10 colunas (Figura 2), e informa que nele o cavalo vai de C a S em 3 pulos. Esse exemplo é usado ao longo do relatório para explicar e conferir o algoritmo.

Cada caso de teste é um arquivo de texto com N linhas de N caracteres: dígitos de 0 a 9, exatamente um C e exatamente um S (isso foi conferido nos oito arquivos). Para cada caso, a saída é o menor número de movimentos de C até S, ou a palavra "impossível" quando S não pode ser alcançada. Os oito tabuleiros vão de 40×40 a 1500×1500 casas, e o maior deles tem mais de dois milhões de casas.

O enunciado não exige recursão, e a solução apresentada usa um laço com uma fila, pelos motivos explicados na Seção 3.

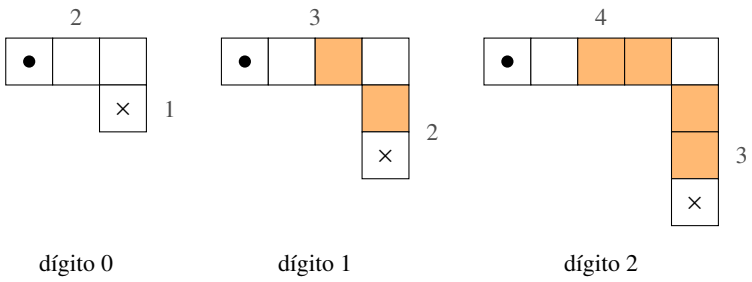


Figura 1 – Forma do pulo para os dígitos 0, 1 e 2. As casas brancas formam o L do dígito 0, o pulo normal de xadrez. Para um dígito d, acrescentam-se d casas (coloridas) a cada perna: nenhuma para $d = 0$, uma para $d = 1$ e duas para $d = 2$. O ponto é a casa de partida, o × é a casa de chegada, e os números indicam o comprimento de cada perna, em passos.

```
   0123456789
0  4286305993
1  67126514C1
2  3541559238
3  6344361236
4  0419867802
5  4271613350
6  S668962685
7  6244051139
8  3714702030
9  0084825587
```

Figura 2 – Tabuleiro de exemplo do enunciado. Linhas e colunas são numeradas de 0 a 9, a partir do canto superior esquerdo: C está na linha 1, coluna 8, e S na linha 6, coluna 0.

Este relatório está organizado da seguinte forma: a Seção 2 mostra como o problema foi modelado, incluindo a regra do pulo e o tratamento do toro; a Seção 3 apresenta o algoritmo, com um exemplo passo a passo; a Seção 4 analisa a eficiência; a Seção 5 mostra os resultados dos oito casos e como eles foram conferidos; e a Seção 6 traz as conclusões.

2 Modelagem

2.1 O tabuleiro como um grafo

Para tratar o problema com algoritmos conhecidos, o tabuleiro é visto como um grafo. Cada casa é um vértice, e há uma aresta da casa A para a casa B quando o cavalo consegue ir de A até B em um pulo. Como cada pulo conta como um movimento, todas as arestas têm o mesmo custo, 1. Assim, o menor número de movimentos de C até S é o menor número de arestas de um caminho de C até S. Esse é o problema do *menor caminho em um grafo sem pesos*.

O grafo é dirigido: como o tamanho do pulo depende do dígito de uma casa, pode haver aresta de A para B sem haver de B para A. Também não se constrói o grafo na memória. Ele é *implícito*: dada uma casa, calculam-se na hora as casas para onde ela leva, a partir do seu dígito. Isso evita guardar as arestas, que somam até $8N^2$ (18 milhões no maior tabuleiro).

A busca em largura (BFS, do inglês *breadth-first search*) resolve exatamente esse problema. Ela visita

as casas por camadas: primeiro a casa C, a distância 0; depois todas as casas a 1 pulo de C; depois as casas a 2 pulos; e assim por diante. Uma fila garante essa ordem, porque as casas descobertas a partir da camada k entram na fila depois de todas as casas da camada k, e portanto só são visitadas depois delas. Por isso, a primeira vez que a busca chega a S é pelo caminho mais curto: se houvesse um caminho com menos pulos, S estaria em uma camada anterior e já teria sido encontrada.

2.2 O tabuleiro toroidal

Para dar a volta nas bordas, usa-se aritmética modular. Se o cavalo está na linha l e na coluna c e faz um pulo que soma a às linhas e b às colunas, ele chega à linha $(l + a) \bmod N$ e à coluna $(c + b) \bmod N$. Isso vale mesmo quando o pulo é maior que o tabuleiro inteiro.

Por exemplo, no tabuleiro 10×10 do enunciado, uma casa com dígito 9 faz um pulo de pernas 10 e 11. Como $10 \bmod 10 = 0$ e $11 \bmod 10 = 1$, o cavalo volta para a mesma linha e chega à coluna seguinte: dá uma volta completa no tabuleiro e termina ao lado de onde saiu.

2.3 A regra do pulo

O enunciado diz que o pulo cresce com o dígito, mas não dá uma fórmula: o tamanho do L aparece só na figura, para os dígitos 0, 1 e 2 (Figura 1). Por isso é preciso extrair o tamanho do L da própria figura. Nela, o L do dígito 0 tem pernas de 2 e 1 casas, o do dígito 1 tem pernas de 3 e 2, e o do dígito 2 tem pernas de 4 e 3. As casas coloridas são o que foi acrescentado: para o dígito d, são d casas em cada perna. Conclui-se que um dígito d forma um L de pernas $1 + d$ e $2 + d$. A figura mostra apenas os dígitos 0, 1 e 2; para os dígitos de 3 a 9, que aparecem nos tabuleiros, o mesmo padrão é estendido. Combinando os sinais e trocando qual perna vai na linha e qual vai na coluna, cada casa tem oito pulos possíveis: $(\pm(1 + d), \pm(2 + d))$ e $(\pm(2 + d), \pm(1 + d))$. Com $d = 0$ recupera-se o pulo normal de xadrez, como o enunciado pede.

O enunciado diz que vale o número da casa onde o cavalo está: um 0 nessa casa é o pulo normal de xadrez, e números maiores esticam o pulo. Portanto é o dígito da casa de partida de cada pulo que define o L, e não o da casa de chegada. A casa C não tem número, e o enunciado manda o cavalo sair dela com um pulo comum de xadrez, o que equivale ao dígito 0. A casa S não influi: o cavalo para ao chegar nela. Assim, cada casa tem um dígito que fixa as oito casas para onde o cavalo pode ir a partir dela, e é isso que define as arestas do grafo da Seção 2.1.

Essa regra foi conferida no exemplo do enunciado, em que C chega a S em 3 pulos: com pernas $(1 + d, 2 + d)$ o resultado é exatamente 3 pulos. Também foram testadas duas outras regras para as pernas, para ver se o exemplo e a figura as distinguem (Tabela 1).

Tabela 1 – Regras testadas para as pernas do L, no tabuleiro de exemplo, com o dígito 0 em C.

Pernas para o dígito d	**Pulos de C a S no exemplo**	**Compatível com a figura?**
$(1 + d,\ 2 + d)$	3	sim
$(2,\ 1 + d)$ ou $(1 + d,\ 2)$	3	não
$(1,\ 2 + d)$ ou $(2 + d,\ 1)$	3	não

As três regras dão 3 pulos no exemplo, de modo que o exemplo não as distingue; quem decide é a figura. A segunda daria, para $d = 1$, pernas iguais, 2 e 2, que não formam um L. Na terceira só uma perna cresce, o que também contraria a figura. Só a primeira regra é compatível com a figura, e é a adotada.

3 Algoritmo

3.1 Escolha do método

Como visto na Seção 2.1, o problema é achar um menor caminho em um grafo em que todas as arestas custam 1. Três métodos clássicos foram considerados:

- **Busca em profundidade (DFS).** Segue um caminho até o fim antes de tentar outro. Ela acha *um* caminho até S, mas não garante que seja o mais curto; para garantir, seria preciso testar todos os caminhos possíveis, o que é inviável. Além disso, se fosse escrita com recursão, a profundidade poderia chegar a N^2 chamadas aninhadas (mais de dois milhões no maior tabuleiro), e a pilha de execução não suporta isso: no Trabalho 1, a pilha já estourou com algumas dezenas de milhares de chamadas.
- **Algoritmo de Dijkstra.** Acha menores caminhos quando as arestas têm custos diferentes, usando uma fila de prioridade. Aqui todos os custos são iguais, então esse trabalho extra é desnecessário: a BFS obtém o mesmo resultado com menos trabalho, $O(V + E)$ contra $O((V + E) \log V)$. O Dijkstra é usado apenas como conferência independente dos resultados (Seção 5).
- **Busca em largura (BFS).** Explora por camadas, e uma fila simples basta. É o método adotado.

Como o enunciado não exige recursão, a BFS é escrita com um laço e uma fila, e não com chamadas recursivas. Isso evita o limite da pilha e deixa o código mais direto: a própria fila guarda o que ainda falta visitar. A escolha da fila também importa: trocá-la por uma pilha, que retira primeiro a casa descoberta mais recentemente, transformaria o laço em uma DFS, e a primeira chegada a S deixaria de garantir o menor caminho. Com a fila, as casas saem na ordem em que foram descobertas, isto é, camada por camada.

3.2 O algoritmo

O Algoritmo 1 mostra a busca, e o Algoritmo 2 mostra o cálculo dos pulos de uma casa. Nas linhas, (l, c) é uma casa (linha e coluna) e N é o lado do tabuleiro.

Como funciona. As linhas 2 a 6 preparam a busca. A matriz $dist$ guarda, para cada casa, quantos pulos foram necessários para chegar a ela, e o valor -1 significa "ainda não descoberta" (linha 4). A casa C começa com distância 0 e é a única na fila (linhas 5 e 6).

O laço das linhas 7 a 17 é o coração do algoritmo. A cada volta, ele retira da fila a casa mais antiga (linha 8). Se for S, a distância dela é a resposta (linhas 9 e 10): pelo argumento da Seção 2.1, a primeira vez que S sai da fila já é pelo caminho mais curto. Se não for, o algoritmo pega o dígito da casa (linha 12) e percorre os 8 pulos que esse dígito permite (linha 13). Para cada pulo, calcula a casa de chegada com aritmética modular, que dá a volta no toro (linha 14); se essa casa ainda não foi descoberta (linha 15), marca a distância dela, que é a da casa atual mais um (linha 16), e a coloca no fim da fila (linha 17). Se a fila esvazia sem que S apareça, todas as casas alcançáveis já foram visitadas e S não está entre elas: a resposta é -1, que o programa apresenta como "impossível" (linha 21).

3.3 Decisões de implementação

- **A matriz $dist$ faz dois papéis.** Ela guarda a distância de cada casa e, ao mesmo tempo, marca quais casas já foram descobertas (-1 quando não foram). Assim não é preciso uma segunda estrutura de visitadas, e o teste da linha 15 leva tempo constante.

Algoritmo 1 Menor número de movimentos do cavalo de C até S

```
 1: função MenorNumeroDeMovimentos(T)
 2:     N ← lado do tabuleiro T
 3:     (l_C, c_C) ← posição de C em T;   (l_S, c_S) ← posição de S em T
 4:     dist[l][c] ← −1 para todas as casas
 5:     dist[l_C][c_C] ← 0
 6:     fila ← fila vazia;   insere (l_C, c_C) na fila
 7:     enquanto fila não está vazia faça
 8:         (l, c) ← retira o primeiro elemento da fila
 9:         se (l, c) = (l_S, c_S) então
10:             devolve dist[l][c]
11:         fim se
12:         d ← dígito da casa (l, c)                    ▷ 0 se for a casa C
13:         para cada (a, b) em Pulos(d) faça
14:             l′ ← (l + a) mod N;   c′ ← (c + b) mod N
15:             se dist[l′][c′] = −1 então
16:                 dist[l′][c′] ← dist[l][c] + 1
17:                 insere (l′, c′) na fila
18:             fim se
19:         fim para
20:     fim enquanto
21:     devolve −1                                       ▷ S não pode ser alcançada
22: fim função
```

Algoritmo 2 Pulos possíveis a partir de uma casa com dígito d

```
1: função Pulos(d)
2:     p ← 1 + d;   q ← 2 + d
3:     devolve {(±p, ±q), (±q, ±p)}                     ▷ 8 pares, combinando os sinais
4: fim função
```

- **A casa é marcada ao entrar na fila, não ao sair.** Se a marcação fosse só ao sair, uma mesma casa poderia entrar na fila várias vezes, uma para cada vizinho que a descobrisse, e o trabalho aumentaria. Marcando na linha 16, cada casa entra no máximo uma vez, e é isso que limita o custo total (Seção 4).
- **A fila é um `ArrayDeque` do Java**, que insere no fim e retira do início em tempo constante.
- **Usa-se `Math.floorMod` em vez do operador `%`.** Em Java, `%` devolve resultado negativo para números negativos (por exemplo, $-2 \bmod 10$ dá -2, e não 8), e as coordenadas de chegada podem ficar negativas quando o pulo vai para cima ou para a esquerda. O `floorMod` devolve sempre um valor entre 0 e $N - 1$.
- **O dígito da casa C (linha 12) vale 0**, conforme a regra do pulo (Seção 2.3). A casa S nunca é expandida, porque o laço termina ao retirá-la.
- **A busca termina ao retirar S da fila (linha 9).** Poderia terminar antes, ao descobrir S; esse ganho foi medido na Seção 6.

3.4 Exemplo passo a passo

O algoritmo foi aplicado ao tabuleiro da Figura 2. A BFS descobre as casas em camadas: 1 casa a 0 pulos (a própria C), 8 casas a 1 pulo, 36 a 2 pulos, 42 a 3 pulos e 13 a 4 pulos, num total de 100, ou seja,

todas as casas do tabuleiro são alcançáveis. A casa S está na camada 3. Ela é retirada da fila depois que 67 casas foram retiradas (contando a própria S), e a busca termina.

Para saber o caminho, anda-se de S para trás: em cada camada procura-se uma casa cujo pulo chega na casa seguinte. Um dos caminhos mínimos é $C(1,8) \to (2,6) \to (2,7) \to S(6,0)$, detalhado na Tabela 2.

Tabela 2 – Um caminho mínimo no tabuleiro de exemplo, pulo a pulo. As coordenadas são (linha, coluna) e o tabuleiro tem $N = 10$.

Pulo	Sai de	Dígito	Pernas	Deslocamento	Sem o módulo	Chega em
1	$(1,8)$	0	$(1,2)$	$(+1,-2)$	$(2,6)$	$(2,6)$
2	$(2,6)$	9	$(10,11)$	$(+10,+11)$	$(12,17)$	$(2,7)$
3	$(2,7)$	2	$(3,4)$	$(+4,+3)$	$(6,10)$	$(6,0)$

O primeiro pulo sai de C e, pela regra da Seção 2.3, é um pulo normal de cavalo. O segundo mostra o toro em ação: o dígito 9 dá pernas de 10 e 11, que em um tabuleiro de lado 10 equivalem a 0 e 1, e o cavalo chega à casa vizinha da direita. O terceiro sai pela borda direita (coluna 10) e entra pela esquerda (coluna 0), caindo em S. A Figura 3 mostra as camadas e o caminho no tabuleiro.

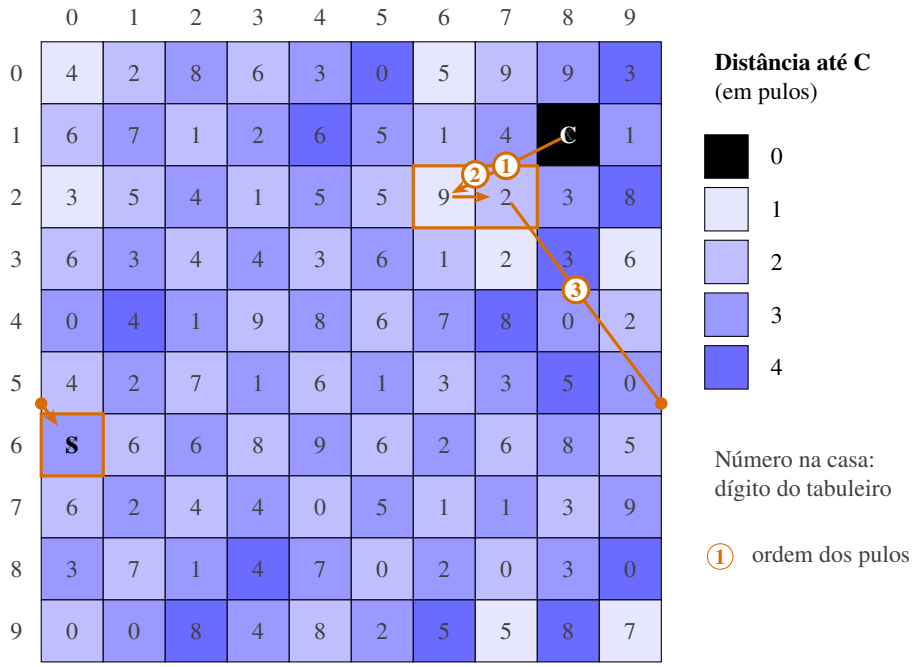


Figura 3 – Camadas da BFS no tabuleiro de exemplo e um caminho mínimo de C a S. A cor indica a distância até C, e as setas numeradas são os três pulos da Tabela 2. O pulo 3 sai pela borda direita e entra pela esquerda.

4 Eficiência

4.1 Análise do custo

Seja N o lado do tabuleiro. Há $V = N^2$ casas, e cada casa tem no máximo 8 pulos, então o grafo tem $E \le 8N^2$ arestas.

Tempo. O trabalho do Algoritmo 1 é contado por partes:

- A inicialização de *dist* (linha 4) percorre todas as casas: $\Theta(N^2)$.

- Cada casa entra na fila no máximo uma vez, porque só entra se *dist* ainda vale -1 (linha 15) e é marcada na hora (linha 16). Logo, o laço da linha 7 roda no máximo N^2 vezes.
- A cada volta, o laço da linha 13 executa exatamente 8 vezes (o Algoritmo 2 sempre devolve 8 pares), e cada execução faz um número constante de operações: duas somas, dois módulos, uma consulta e uma atualização em *dist* e uma inserção na fila, todas em tempo constante.

Somando, o laço principal faz no máximo $N^2 \times 8 \times c$ operações, para uma constante c, ou seja, $O(N^2)$. Em termos de vértices e arestas, é o custo $\Theta(V + E)$ conhecido da BFS. Como a inicialização já custa N^2 em qualquer tabuleiro, o tempo do algoritmo é $\Theta(N^2)$ sempre. O que muda de um tabuleiro para outro é a constante: quando S é encontrada cedo, a busca visita só parte das casas.

Espaço. A matriz *dist* ocupa N^2 inteiros, a fila guarda no máximo N^2 casas (cada uma entra uma vez) e o próprio tabuleiro ocupa N^2 caracteres. O espaço é, portanto, $\Theta(N^2)$.

Não dá para melhorar a *ordem de grandeza*: só para localizar S no tabuleiro, qualquer algoritmo precisa, no pior caso, olhar as N^2 casas. A BFS é, então, assintoticamente ótima para este problema, e os ganhos que restam são de constante (Seção 6).

4.2 Conferência com as medições

Se o custo é proporcional a N^2, dobrar N deve multiplicar o tempo por 4. Para conferir, mede-se o pior caso: uma busca que não para em S e visita todas as casas alcançáveis. Isso isola o efeito do tamanho do tabuleiro, sem depender de onde S está.

As medições foram feitas em um Apple M2 com 16 GB de memória, macOS 26.7 e Java 24.0.1, sem contar a leitura do arquivo. Cada tempo é o melhor de 15 execuções na mesma JVM, o que descarta o custo de compilação das primeiras execuções e a interferência da coleta de lixo. O tamanho da memória da JVM foi fixado (`-Xms2g -Xmx2g`), mas mesmo assim o tempo absoluto por casa visitada variou de uma execução para outra, de cerca de 55 a 108 ns nas execuções feitas. As razões entre tamanhos, que são o que se usa aqui, praticamente não mudaram: de 800 para 1500, por exemplo, deu 3,6 em uma execução e 3,9 em outra, para 3,5 esperado.

Tabela 3 – Exploração completa, sem parar em S. As duas últimas colunas comparam cada tamanho com o anterior: se o custo é proporcional a N^2, elas devem ser parecidas.

N	**Casas alcançadas**	**Tempo (ms)**	**ns por casa**	**Tempo / anterior**	**N^2 / anterior**
40	1.600	0,14	88	-	-
80	6.398	0,62	96	4,4	4,0
100	9.999	0,97	97	1,6	1,6
150	22.494	2,23	99	2,3	2,3
200	39.992	4,02	101	1,8	1,8
400	159.950	16,41	103	4,1	4,0
800	639.857	67,35	105	4,1	4,0
1500	2.249.516	241,98	108	3,6	3,5

A Tabela 3 confirma a previsão: a razão entre os tempos acompanha a razão entre os valores de N^2 em todos os tamanhos. O tempo por casa visitada, que deveria ser constante, sobe pouco, de 88 para 108 ns (23%), enquanto o número de casas cresce 1400 vezes. Uma hipótese para essa pequena alta é o uso de memória: o maior tabuleiro ocupa dezenas de megabytes, que não cabem nos caches do processador; isso não foi medido. Para $N \leq 100$ os tempos ficam abaixo de 1 ms, perto do limite do que o cronômetro e a JVM medem com precisão, e as razões desses tamanhos devem ser lidas com cuidado.

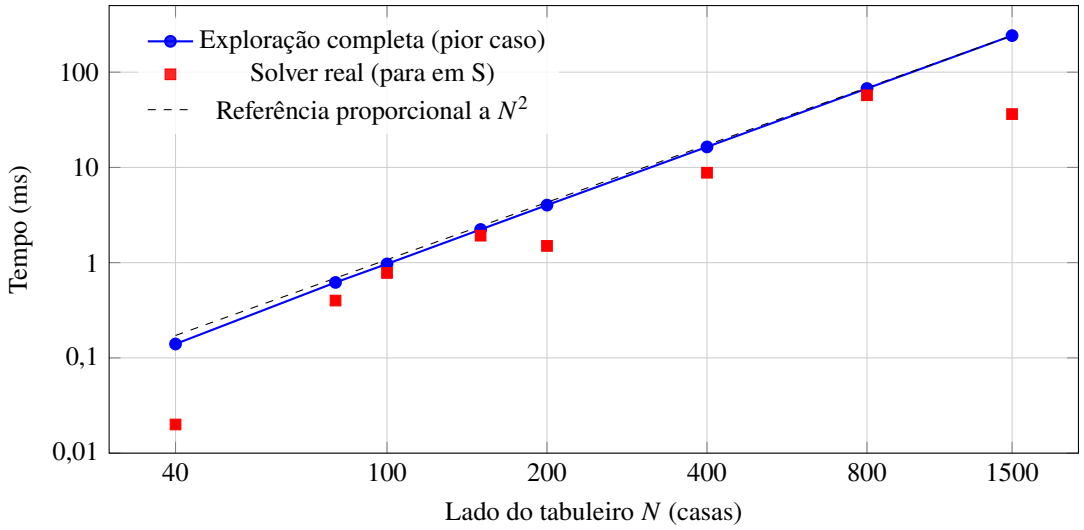


Figura 4 – Tempo de execução por tamanho do tabuleiro, em escala logarítmica nos dois eixos. A exploração completa acompanha a reta tracejada, proporcional a N^2, que passa pelo ponto $N = 1500$. Os pontos vermelhos são os casos reais, em que a busca para ao chegar a S.

4.3 Por que os casos reais não crescem de forma regular

Nos casos reais o algoritmo para ao retirar S da fila, e o trabalho depende de onde S está em relação a C. A Tabela 4 mostra que a busca retira da fila desde 16,7% das casas (caso 1500) até 93,3% (caso 800). Por isso o caso 1500, que tem 3,5 vezes mais casas que o caso 800, é resolvido mais rápido (36,30 ms contra 57,24 ms): a BFS chega a S depois de visitar uma parte pequena do tabuleiro, e o resto nem chega a ser explorado. É por isso que os pontos vermelhos da Figura 4 ficam abaixo da curva do pior caso e não seguem uma reta.

5 Resultados

Aplicando o algoritmo da Seção 3 aos oito casos de teste, com a regra do pulo da Seção 2.3, obtêm-se os resultados da Tabela 4.

Tabela 4 – Resultados dos oito casos de teste. A coluna "Casas retiradas" conta as casas que saíram da fila até a retirada de S, e o tempo é o melhor de 15 execuções.

Caso	N	Movimentos	Casas retiradas	% de N^2	Tempo (ms)
caso40	40	4	425	26,6	0,02
caso80	80	6	4.483	70,0	0,40
caso100	100	8	8.596	86,0	0,78
caso150	150	11	20.863	92,7	1,92
caso200	200	9	15.483	38,7	1,50
caso400	400	19	92.523	57,8	8,79
caso800	800	43	597.282	93,3	57,24
caso1500	1500	34	375.230	16,7	36,30

Nenhum dos oito casos é impossível: em todos, S é alcançável a partir de C. O número de movimentos vai de 4 a 43, e o tabuleiro de maior lado, 1500×1500, foi resolvido em 36 ms. O caso mais demorado é o 800, com 57 ms, e a razão disso, que não é o tamanho, está na Seção 4.3.

5.1 Como os resultados foram conferidos

Para não confiar em um único programa, os resultados foram conferidos de várias formas:

- **Exemplo do enunciado.** O algoritmo dá 3 pulos, como o enunciado informa, e uma busca exaustiva confirma que 2 pulos não bastam.
- **Segunda implementação.** A busca foi escrita de novo, separadamente (casas guardadas como números inteiros e fila em vetor), para que um erro do primeiro programa não se repetisse igual na conferência. As duas dão a mesma resposta nos oito casos.
- **Algoritmo de Dijkstra.** Um método diferente, com fila de prioridade, dá a mesma distância nos casos até 400×400.
- **Caminho legal.** Reconstruiu-se o caminho mínimo de cada caso e conferiu-se, pulo a pulo, que todos obedecem à regra do pulo.
- **Tabuleiros pequenos aleatórios.** Em 389 tabuleiros sorteados (com semente fixa), a distância encontrada é atingível e uma busca exaustiva em profundidade limitada não acha caminho menor.
- **Casos de borda.** Tabuleiros construídos à mão testam o pulo padrão, um pulo que só fecha dando a volta no toro e o dígito 9 em um tabuleiro 7×7 (pernas maiores que o lado, o que testa o módulo). A saída "impossível" foi exercitada com um tabuleiro sem solução, e a busca independente também não achou caminho.

Essas conferências mostram que o programa implementa corretamente a regra do pulo da Seção 2.3. Como as implementações independentes seguem a mesma regra, elas testam a implementação, e não a interpretação do enunciado.

6 Conclusão

O problema foi resolvido modelando-se o tabuleiro como um grafo implícito, com uma casa por vértice e um pulo por aresta, e usando busca em largura. A modelagem transformou "menor número de movimentos" em "menor caminho em um grafo sem pesos", problema para o qual a BFS é correta e custa $\Theta(N^2)$, o que é ótimo em ordem de grandeza. Os oito casos foram resolvidos, com 4 a 43 movimentos e no máximo 57 ms, e os resultados foram conferidos de várias formas (Seção 5.1). As medições confirmaram o custo quadrático: dobrar N multiplica o tempo por cerca de 4 (Tabela 3).

Os resultados trazem duas observações. A primeira é que o número de movimentos não cresce com o tamanho do tabuleiro: o caso 800 precisa de 43 movimentos e o caso 1500, de 34 (Tabela 4). A distância depende de onde estão C e S e de quais dígitos aparecem no caminho, e não só de N, e isso se soma ao efeito de a busca parar em S para tornar o tempo de cada caso irregular (Seção 4.3). A segunda é que os tabuleiros são quase totalmente conectados: a exploração completa da Seção 4.2 alcança mais de 99,9% das casas em todos os casos, e por isso nenhum caso real deu "impossível", embora o algoritmo trate essa saída.

6.1 Limitações

Há três limites que convém ter presentes ao ler os resultados. O primeiro é a regra do pulo para os dígitos de 3 a 9: a figura do enunciado mostra apenas os dígitos 0, 1 e 2, e o padrão foi estendido aos demais (Seção 2.3). A segunda implementação e as demais conferências (Seção 5.1) seguem a mesma regra, de modo que testam o código, não essa extensão. O segundo é que as medições vêm de um único

computador e de uma única versão da máquina virtual Java: o tempo absoluto variou entre execuções, e só as razões entre tamanhos são estáveis (Seção 4.2). O terceiro é que o crescimento quadrático foi conferido apenas na faixa dos oito casos, de $N = 40$ a $N = 1500$; em tabuleiros bem maiores, o uso de memória pode passar a pesar mais que a contagem de operações.

6.2 Melhorias possíveis

Mesmo sendo ótimo em ordem de grandeza, o algoritmo pode ficar mais rápido por um fator constante. Duas melhorias foram implementadas e medidas, sempre conferindo que a resposta continua a mesma nos oito casos e no exemplo.

Parar ao descobrir S (versão B). A BFS atribui a uma casa a sua distância definitiva no momento em que a descobre. Portanto não é preciso esperar S chegar ao início da fila para devolver a resposta, como é feito na linha 9: basta testar ao inserir (linha 17). Isso evita expandir as casas da última camada que ainda estão na fila antes de S. A versão B foi de 1,5 a 1,9 vez mais rápida que a original nos casos com $N \geq 100$ (Tabela 5).

Casas como inteiros e pulos sem alocação (versão C). Na versão original, cada casa visitada aloca cerca de dez vetores pequenos: um par (l, c) para entrar na fila e os 8 pares de deslocamento devolvidos por PULOS. A versão C, que também para ao descobrir S, guarda a casa como um único inteiro $l \times N + c$, usa um vetor de inteiros como fila e calcula os 8 pulos no próprio laço, sem criar objetos. Ela foi de 2,9 a 4,4 vezes mais rápida que a original. Acredita-se que o ganho venha de evitar as alocações e de acessar a memória de forma mais compacta, mas cada efeito não foi isolado. O preço é um código menos legível: os oito pulos saem de operações sobre bits do índice do laço, e não de uma lista de pares como no Algoritmo 2, e uma mudança na regra do pulo exigiria mexer nesse laço.

Tabela 5 – Tempo das três versões, em ms (melhor de 15 execuções), e o quanto a versão original (A) é mais lenta que as outras. Os dois menores casos ($N = 40$ e $N = 80$) ficam abaixo de 0,5 ms e foram omitidos pelo ruído de medição. A medição é independente da Tabela 4, então a versão A difere dela em alguns por cento.

N	A: original	B: para ao descobrir S	C: inteiros e B	A / B	A / C
100	0,75	0,39	0,17	1,9	4,4
150	1,94	1,09	0,48	1,8	4,0
200	1,36	0,75	0,34	1,8	4,0
400	8,77	5,12	2,43	1,7	3,6
800	57,95	38,26	16,39	1,5	3,5
1500	36,36	23,25	12,61	1,6	2,9

Três outras ideias não foram implementadas. A primeira é a busca bidirecional, a partir de C e de S ao mesmo tempo. Aqui ela não é imediata: como o pulo depende da casa de onde se sai, andar para trás a partir de S exige saber quais casas chegam a cada casa, e isso obriga a montar o grafo inverso, que custa $\Theta(N^2)$ de tempo e até $8N^2$ entradas de memória. Não se sabe se o ganho compensaria esse custo; seria preciso medir. A segunda vale se o mesmo tabuleiro tivesse várias saídas: a exploração completa da Seção 4.2 já calcula a distância de C a todas as casas, então uma única busca responderia a todas. A terceira reduz a memória: como a resposta só depende do número da camada em que S aparece, não é preciso guardar a distância de cada casa. A matriz $dist$ de inteiros poderia ser trocada por um vetor de bits que apenas marca as casas já descobertas, mais as listas das casas da camada atual e da seguinte. No maior tabuleiro isso troca cerca de 9 MB de inteiros por cerca de 280 KB de bits, mais as duas listas. Se a hipótese da Seção 4.2 sobre os caches estiver certa, isso também poderia reduzir a alta do tempo por casa nos tabuleiros grandes. Esta ideia não foi implementada nem medida.

Por fim, o que mais pesou neste trabalho não foi o algoritmo, que é padrão, e sim a modelagem: decidir o que é vértice e o que é aresta, como tratar o toro e como extrair da figura do enunciado a regra do pulo.

Foram essas decisões que definiram os resultados, e por isso buscou-se registrar cada uma com os seus motivos.